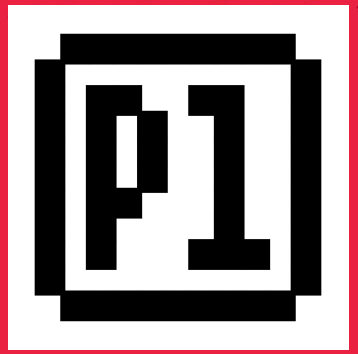
Operaciones con matrices



UNRN

Universidad Nacional
de **Río Negro**

r33

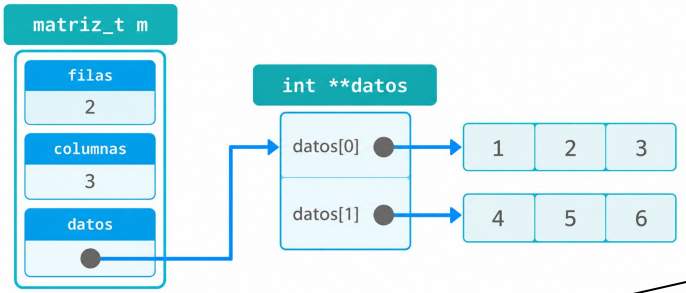
21 / 2026

Operaciones con matrices

UNRN Universidad Nacional de Río Negro

Matriz dinamica

```c
typedef struct {
    int filas;
    int columnas;
    int **datos;
} matriz_t;
```

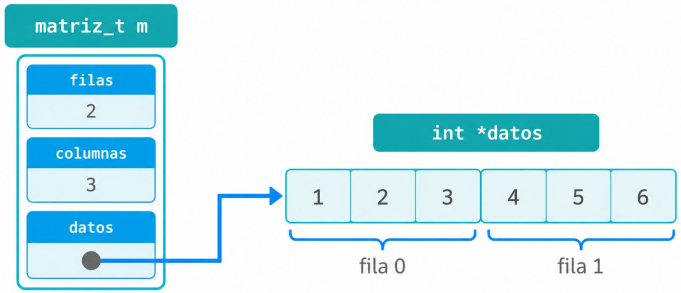


	0	1	2
0	A	B	C
1	D	E	F
2	G	H	I

```c
m->datos[i][j]
```

Matriz dinamica

```c
typedef struct {
    int filas;
    int columnas;
    int *datos;
} matriz_t;
```



	0	1	2
0	A	B	C
1	D	E	F
2	G	H	I

```c
m->datos[i * m->columnas + j]
```

Transposición

	0	1	2
0	A	B	C
1	D	E	F
2	G	H	I

$$T[j][i] = A[i][j]$$

	0	1	2
0	A	D	G
1	B	E	H
2	C	F	I

Matriz Transpuesta

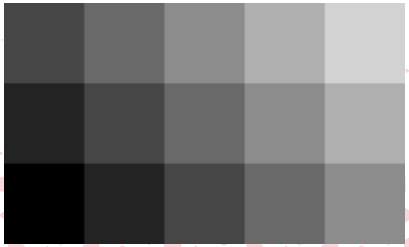
	0	1	2	3	4
0	A	B	C	D	E
0	F	G	H	I	J

	0	0
0	A	F
1	B	G
2	C	H
3	D	I
4	E	J

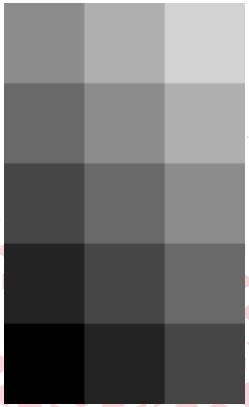
No necesariamente tiene que ser cuadrada

Matriz Transpuesta

	0	1	2	3	4
0	A	B	C	D	E



	0
0	A
1	B
2	C
3	D
4	E



T[i][j] = A[j][i]

```
for (int i = 0; i < N; i++) //O(N)
{
    for (int j = 0; j < M; j++) //O(M)
    {
        T[j][i] = A[i][j];    //O(1)
    }
}
```

complejidad:

O(N)*O(M)*O(1)= O(N*M)

si N=M

=O(N^2)

doble puntero simple

```
for (int i = 0; i < N; i++) // O(N)
{
        for (int j = 0; j < M; j++) // O(M)
      {
            T->datos[j][i] = A->datos[i][j]; //
O(1)
         }
   }
```

indice= [i][j]

complejidad:

O(N)*O(M)*O(1)= O(N*M)

si N=M

=O(N^2)

puntero simple

```
for (int i = 0; i < N; i++) {          // O(N)
        for (int j = 0; j < M; j++) {    // O(M)

            T->datos[j * N + i] =
                A->datos[i * M + j];     // O(1)
        }
    }
```

indice= i* M+j

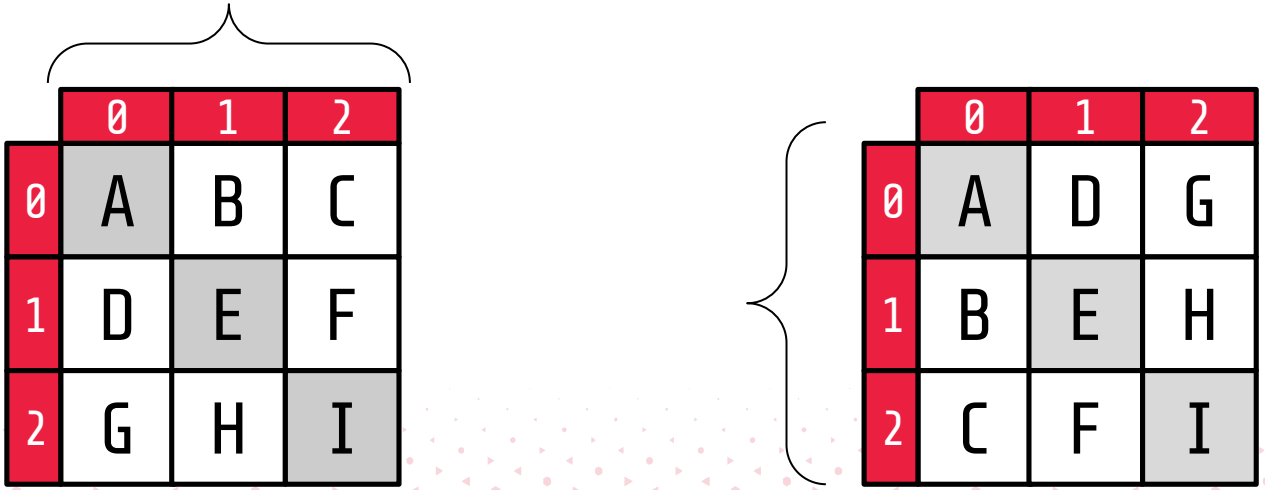
complejidad:

O(N)*O(M)*O(1)= O(N*M)

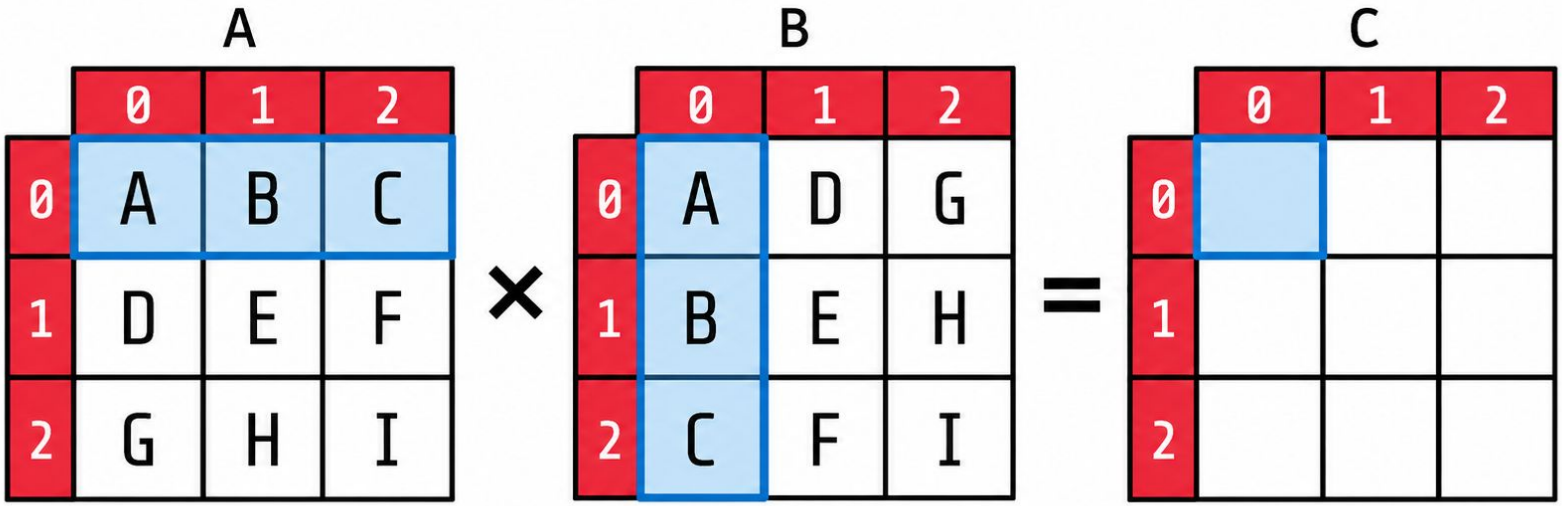
si N=M

=O(N^2)

Multiplicacion

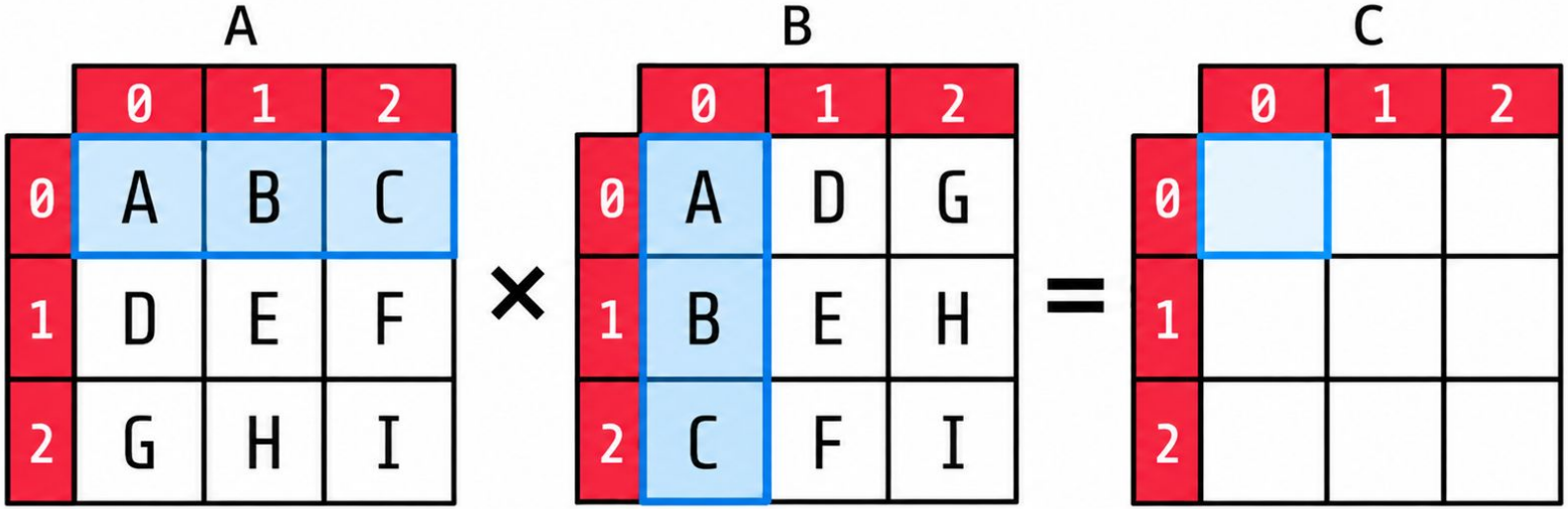


Multiplicación de matrices: A × B



Paso 1: elegir una fila de A y una columna de B.

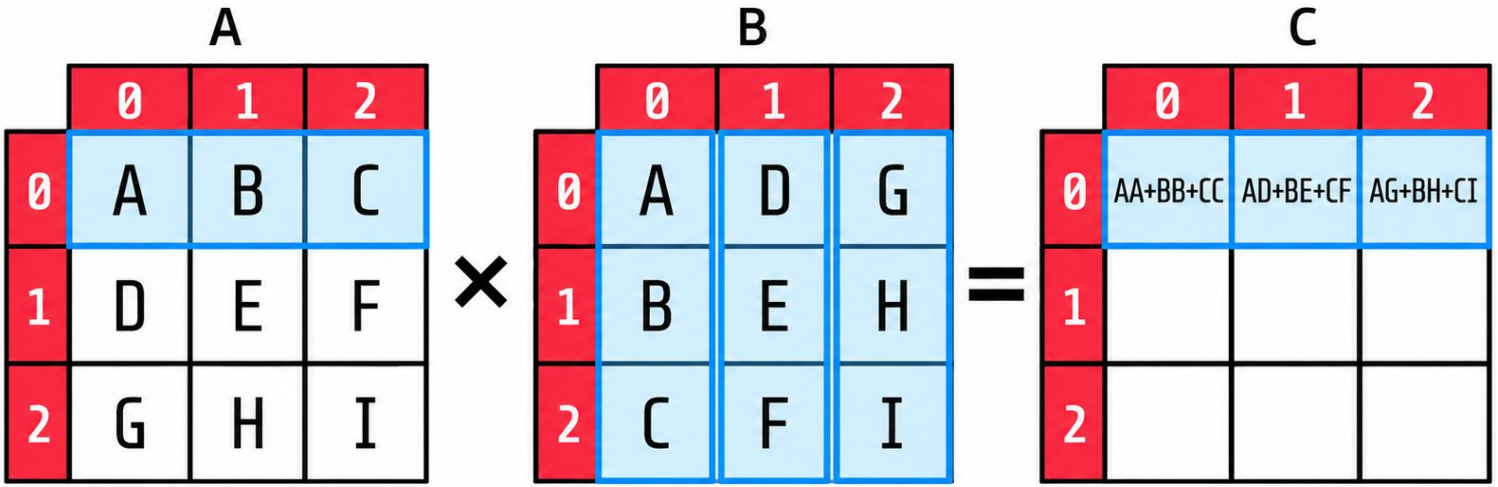
Multiplicación de matrices: A × B



$$C[0][0] = A \cdot A + B \cdot B + C \cdot C$$

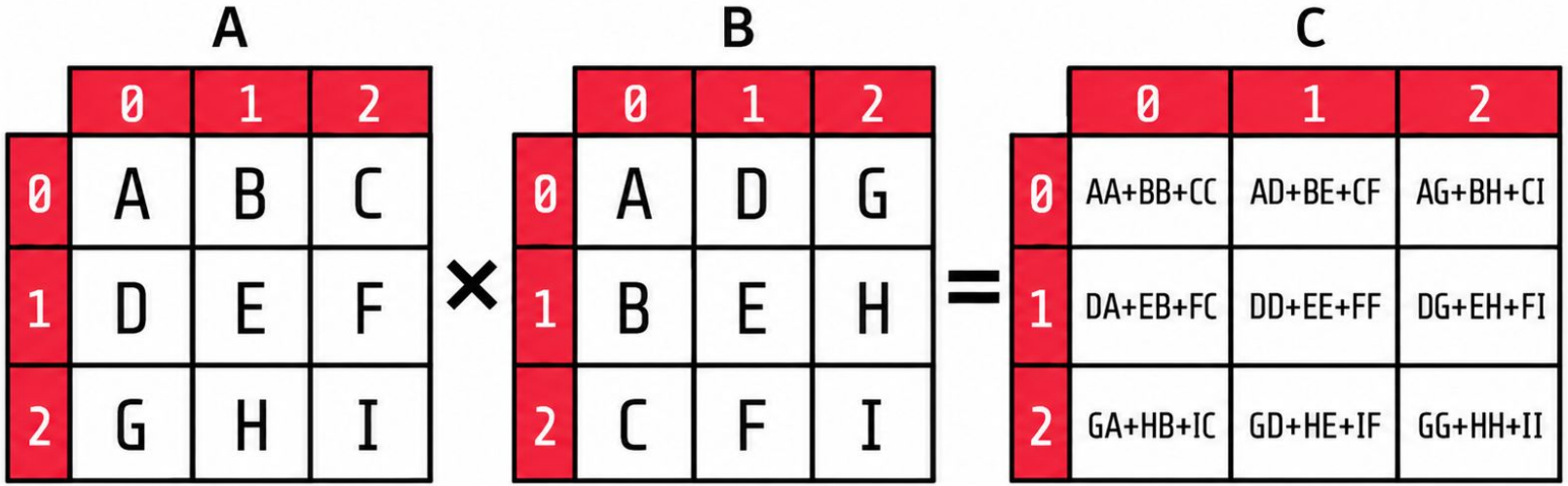
Paso 2: multiplicar fila 0 por columna 0.

Multiplicación de matrices: A × B



Paso 3: repetir con cada columna de B para completar la fila 0 de C.

Multiplicación de matrices: A × B



Paso 4: completar toda la matriz resultado C.

Cada elemento de C se obtiene multiplicando una fila de A por una columna de B.

A*B

```
for (int i = 0; i < N; i++) {           // O(N)
    for (int j = 0; j < P; j++) {       // O(P)

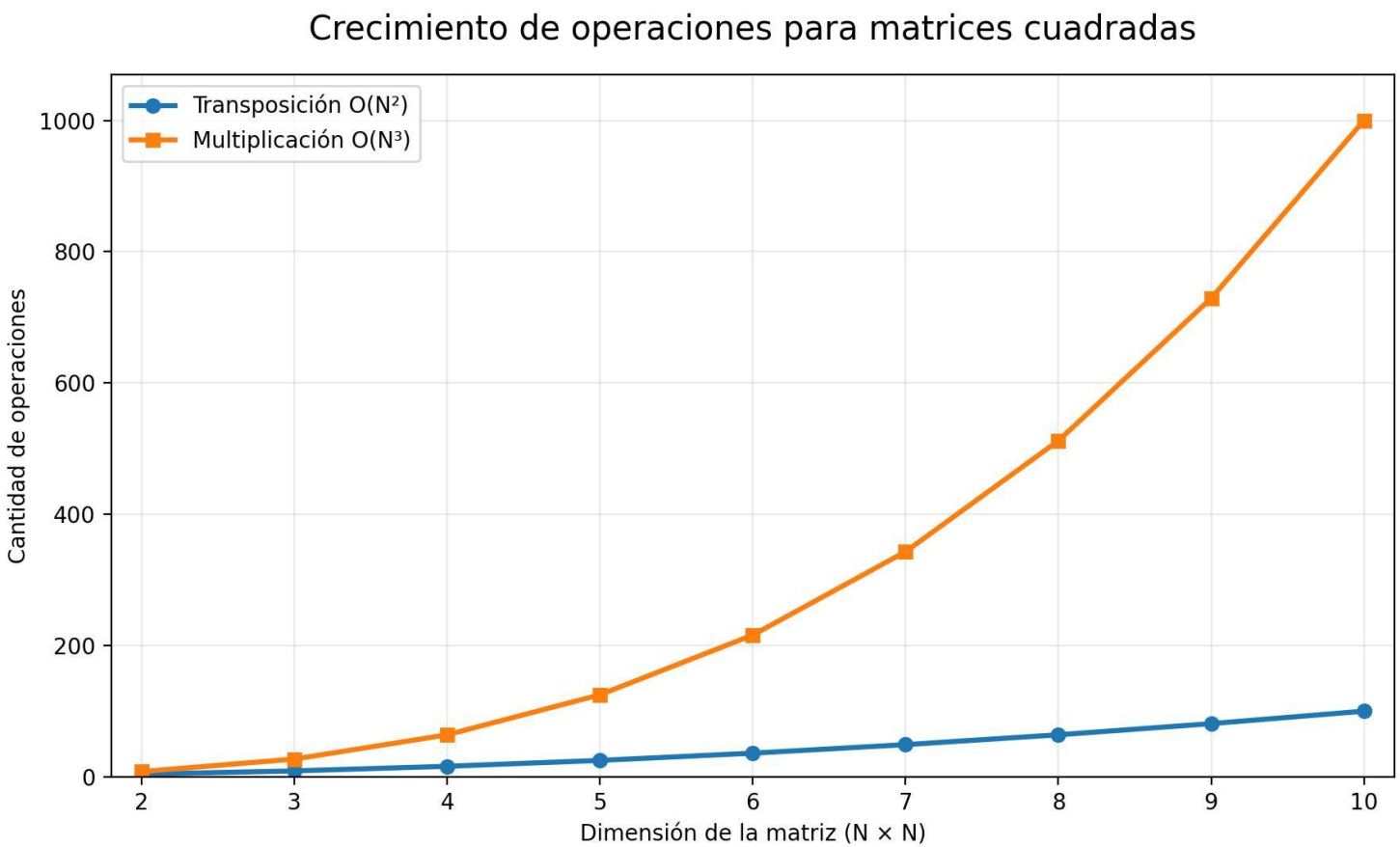
        C->datos[i][j] = 0;

        for (int k = 0; k < M; k++) {   // O(M)
            C->datos[i][j] +=
                A->datos[i][k] * B->datos[k][j];
        }
    }
}
```

complejidad:

O(N)*O(M)*O(P)= O(N*M*P)

=O(N^3)


Crecimiento de operaciones para matrices cuadradas
Transposición O(N²)
Multiplicación O(N³)
Cantidad de operaciones
0
200
400
600
800
1000
2
3
4
5
6
7
8
9
10
Dimensión de la matriz (N × N)

Convolucion^{F_suavizador}

Imagen original (valores de píxeles)

	0	1	2	3	4	5	6
0	10	10	10	10	10	10	10
1	10	50	50	50	50	50	10
2	10	50	200	200	200	50	10
3	10	50	200	200	200	50	10
4	10	50	200	200	200	50	10
5	10	50	50	50	50	50	10
6	10	10	10	10	10	10	10

Tomamos un vecindario 3 × 3 de la imagen alrededor del píxel en la posición (3, 3).

×

Filtro suavizador (promedio 3 × 3)

	0	1	2
0	1/9	1/9	1/9
1	1/9	1/9	1/9
2	1/9	1/9	1/9

Multiplicación elemento a elemento y suma

	0	1	2
0	200 × 1/9	200 × 1/9	200 × 1/9
1	200 × 1/9	200 × 1/9	200 × 1/9
2	200 × 1/9	200 × 1/9	200 × 1/9

Resultado del píxel (3, 3):

$$\frac{200+200+200+200+200+200+200+200+200}{9}$$

$$= \frac{1800}{9} = 200$$

Imagen resultante (más suave)

	0	1	2	3	4	5	6
0	10	18	28	32	28	18	10
1	18	49	83	94	83	49	18
2	28	83	133	150	133	83	28
3	32	94	150	167	150	94	32
4	28	83	133	150	133	83	28
5	18	49	83	94	83	49	18
6	10	18	28	32	28	18	10

El valor central pasó de 200 a 167 debido al promedio con sus vecinos. La imagen se ve más suave (menos abrupta).

Convolucion F_realce

Imagen original (valores de píxeles)

	0	1	2	3	4	5	6
0	10	10	10	10	10	10	10
1	10	50	50	50	50	50	10
2	10	50	200	200	200	50	10
3	10	50	200	200	200	50	10
4	10	50	200	200	200	50	10
5	10	50	50	50	50	50	10
6	10	10	10	10	10	10	10

Tomamos un vecindario 3 × 3 de la imagen alrededor del píxel en la posición (3, 3).

×

Filtro Sobel horizontal (Gx)
(detecta bordes verticales)

	0	1	2
0	-1	0	1
1	-2	0	2
2	-1	0	1

	0	1	2
0	200	200	200
1	200	200	200
2	200	200	200

Multiplicación elemento a elemento y suma (Gx)

	0	1	2
0	200×-1 =-200	200×0 =0	200×1 =200
1	200×-2 =-400	200×0 =0	200×2 =400
2	200×-1 =-200	200×0 =0	200×1 =200

Suma:

-200 + 0 + 200 - 400 + 0 + 400 - 200 + 0 + 200

= 0

Filtro Sobel vertical (Gy)
(detecta bordes horizontales)

	0	1	2
0	-1	-2	-1
1	0	0	0
2	1	2	1

	0	1	2
0	200	200	200
1	200	200	200
2	200	200	200

Multiplicación elemento a elemento y suma (Gy)

	0	1	2
0	200×-1 =-200	200×-2 =-400	200×-1 =-200
1	200×0 =0	200×0 =0	200×0 =0
2	200×1 =200	200×2 =400	200×1 =200

Suma:

-200 - 400 - 200 + 0 + 0 + 0 + 200 + 400 + 200

= 0

Magnitud del gradiente

$$G = \sqrt{G_x{}^2 + G_y{}^2}$$

$$G = \sqrt{0^2 + 0^2} = 0$$

El píxel central tiene un valor bajo en la imagen de bordes porque está en una región uniforme (sin cambio de intensidad).

Ejemplo en un píxel de borde

	0	1	2
0	50	200	200
1	50	200	200
2	50	200	200

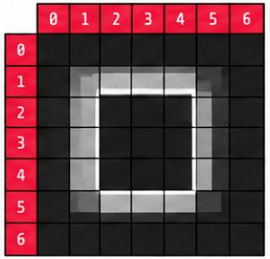
Para este píxel (3, 2):

$G_x = 600$ $\quad G_y = 0$

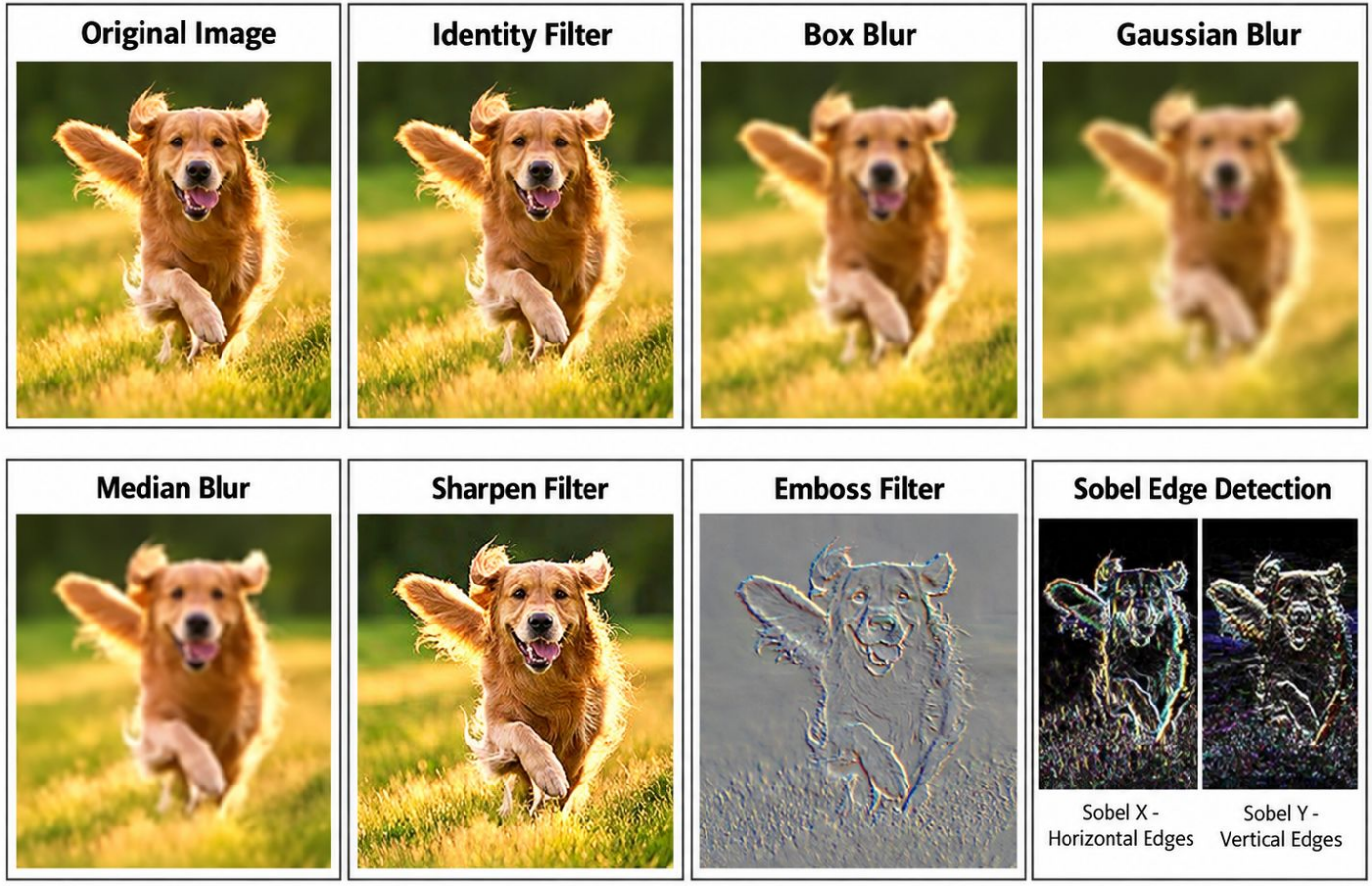
$$G = \sqrt{600^2 + 0^2} = 600$$

Valor alto en la imagen de bordes porque hay un cambio de intensidad.

Imagen resultante de detección de bordes (magnitud de gradiente)



Los valores altos (más claros) indican bordes, donde hay cambios de intensidad en la imagen original. El filtro Sobel detecta bordes verticales (Gx) y horizontales (Gy), y la magnitud del gradiente combina ambas direcciones.


Original Image
Identity Filter
Box Blur
Gaussian Blur
Median Blur
Sharpen Filter
Emboss Filter
Sobel Edge Detection
Sobel X - Horizontal Edges
Sobel Y - Vertical Edges

	0	1	2	3	4
0	A	B	C	D	E
1	F	G	H	I	J
2	K	L	M	N	O

	0	1	2
0	A	B	C
1	F	G	H
2	K	L	M


¿Preguntas?

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar